

Sessional 1 DSA Notes

Sessional 1 — Complete DSA Notes (C++)

1. Introduction to Data Structures & General Programming Concepts

Data Structure = a way of organizing and storing data so it can be accessed and modified efficiently.

Classification:

- **Primitive:** int, char, float, pointer — built into the language.
- **Non-primitive:** Arrays, Linked Lists, Stacks, Queues, Trees, Graphs.
- **Linear** (elements in sequence): Array, Linked List, Stack, Queue.
- **Non-linear** (hierarchical/networked): Trees, Graphs.

Abstract Data Type (ADT) An ADT defines *what* operations a structure supports, not *how* they're implemented. Example — **Stack ADT**: push(), pop(), peek(), isEmpty() — you don't care whether it's built with an array or a linked list, just that these operations behave correctly (LIFO).

Why this matters for exams: you're often asked "give the ADT for X" — just list the operations and their contract (pre/post conditions), not the code.

General Programming Concepts recap

- **Pointer:** a variable that stores a memory address.
- **Array:** contiguous block of memory of the same type, accessed via `base_address + index * sizeof(type)`.

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;           // p points to arr[0]
cout << *(p + 2);       // prints 30 -> equivalent to arr[2]
```

LeetCode equivalent: 1. Two Sum — basic array traversal/indexing.

2. Structures, Pointers & Memory Representation

2.1 Structures — dot . vs arrow ->

```
struct Student {
    int id;
    float gpa;
};

Student s1;
s1.id = 5;           // dot operator: use with an OBJECT (stack variable)
```

```

Student *sp = &s1;
sp->id = 5;           // arrow operator: use with a POINTER to an
object
(*sp).id = 5;         // equivalent to sp->id, just less clean

```

Rule of thumb: `p->member` is literally shorthand for `(*p).member`.

2.2 Row-Major vs Column-Major

For a 2D array `A[ROWS][COLS]` stored starting at base address `B`, with element size `w`:

- **Row-Major** (C/C++ default): rows are stored one after another.
Address(`A[i][j]`) = `B + (i * COLS + j) * w`
- **Column-Major** (Fortran/MATLAB): columns are stored one after another. Address(`A[i][j]`) = `B + (j * ROWS + i) * w`

Example: `int A[3][4]`, base = 1000, `w = 4`, find address of `A[2][1]`
(row-major): $1000 + (2 \cdot 4 + 1) \cdot 4 = 1000 + 36 = 1036$

2.3 Pointers and Arrays

```

int arr[5];
int *p = arr;           // arr decays to a pointer to arr[0]; arr ==
&arr[0]
// &arr is different: it's a pointer to the WHOLE ARRAY (type int(*)
[5])

```

2.4 Pointers with Multi-Dimensional Arrays

```

int A[3][4];
int (*p)[4] = A;        // pointer to an array of 4 ints - must
specify column size
cout << *((p + i) + j); // == A[i][j]

```

2.5 Array of Pointers

```

int a = 1, b = 2, c = 3;
int *arr[3] = {&a, &b, &c}; // array of 3 int-pointers
cout << *arr[1];             // prints 2

```

2.6 Pointers and Strings

```

char *s = "Hello";       // s points to the first char of a string
literal
cout << *(s+1);          // 'e'

```

2.7 Array of Pointers to Strings

```

const char *fruits[] = {"Apple", "Banana", "Cherry"};
cout << fruits[1];        // "Banana"
cout << *(fruits[1] + 2); // 'n' -> 3rd char of "Banana"

```

LeetCode equivalents: [54. Spiral Matrix](#) (2D indexing), [74. Search a 2D Matrix](#).

3. Complexity Analysis, Big-O/ Ω / Θ , Dynamic Memory Allocation

3.1 Asymptotic Notations

Bound type

Notation	Meaning	
Big O (O)	Worst-case growth rate	Upper bound
Big Omega (Ω)	Best-case growth rate	Lower bound
Big Theta (Θ)	Average / tight bound	Both upper & lower

Common growth rates, fastest to slowest: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

3.2 Dynamic Memory Allocation in C++

```
int *p = new int;           // allocate single int on heap
int *arr = new int[10];    // allocate array of 10 ints on heap
delete p;                  // free single object
delete[] arr;              // free array – must use [] or undefined
behavior
```

new (C++) vs malloc (C): new calls the constructor and is type-safe; malloc just gives raw bytes and needs a cast.

3.3 Time Complexity Cheat-Sheet (the part you specifically asked for)

Array (unsorted):

Operation	Complexity	Why
Access by index	$O(1)$	direct address calculation
Search (linear)	$O(n)$	must check each element
Insert at end	$O(1)^*$	*if space available; $O(n)$ if resize needed
Insert at start	$O(n)$	must shift all elements right
Insert at middle	$O(n)$	must shift elements after the position
Delete at end	$O(1)$	no shifting needed
Delete at start	$O(n)$	must shift all elements left
Delete at middle	$O(n)$	must shift elements after the position

Array (sorted):

Operation	Complexity	Why
Search	$O(\log n)$	binary search possible
Insert	$O(n)$	find position $O(\log n)$ + shift $O(n)$
Delete	$O(n)$	find $O(\log n)$ + shift $O(n)$

Singly Linked List:

Operation	Complexity	Why
Access by index	$O(n)$	must traverse from head
Search	$O(n)$	traverse until found
Insert at head (front)	$O(1)$	just relink head pointer
Insert at tail (end)	$O(n)$	must traverse to find last node ($O(1)$ if you keep a tail pointer)
Insert at middle	$O(n)$	must traverse to find position move head to head-

Delete at head	O(1)	>next
Delete at tail	O(n)	must traverse to node before last
Delete at middle	O(n)	must find the node

Stack (array-based or linked-list based):

Operation	Complexity
push	O(1)
pop	O(1)
peek/top	O(1)
isEmpty	O(1)

LeetCode equivalents: [35. Search Insert Position](#) (binary search, O(log n)), [704. Binary Search](#).

4. Linked Lists (Singly) — Full Implementation

4.1 Node Definition

```
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

Node* head = nullptr; // empty list
```

4.2 Insertion at Front — O(1)

```
void insertAtFront(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;
    head = newNode;
}
```

4.3 Insertion at End — O(n)

```
void insertAtEnd(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (head == nullptr) { head = newNode; return; }
    Node* temp = head;
    while (temp->next != nullptr) temp = temp->next;
    temp->next = newNode;
}
```

4.4 Insertion at a Given Position — O(n)

```
void insertAtPosition(Node*& head, int val, int pos) {
    if (pos == 0) { insertAtFront(head, val); return; }
    Node* temp = head;
    for (int i = 0; i < pos - 1 && temp != nullptr; i++) temp = temp->next;
    if (temp == nullptr) return; // invalid position
    Node* newNode = new Node(val);
    newNode->next = temp->next;
    temp->next = newNode;
}
```

4.5 Traversal / Display — $O(n)$

```
void display(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

4.6 Search — $O(n)$

```
bool search(Node* head, int key) {
    Node* temp = head;
    while (temp != nullptr) {
        if (temp->data == key) return true;
        temp = temp->next;
    }
    return false;
}
```

4.7 Deletion at Front — $O(1)$

```
void deleteAtFront(Node*& head) {
    if (head == nullptr) return;
    Node* temp = head;
    head = head->next;
    delete temp;
}
```

4.8 Deletion at End — $O(n)$

```
void deleteAtEnd(Node*& head) {
    if (head == nullptr) return;
    if (head->next == nullptr) { delete head; head = nullptr;
return; }
    Node* temp = head;
    while (temp->next->next != nullptr) temp = temp->next;
    delete temp->next;
    temp->next = nullptr;
}
```

4.9 Deletion by Value — $O(n)$

```
void deleteNode(Node*& head, int val) {
    if (head == nullptr) return;
    if (head->data == val) { deleteAtFront(head); return; }
    Node* temp = head;
    while (temp->next != nullptr && temp->next->data != val) temp =
temp->next;
    if (temp->next == nullptr) return; // not found
    Node* toDelete = temp->next;
    temp->next = toDelete->next;
    delete toDelete;
}
```

4.10 Reversal — Iterative $O(n)$

```
void reverse(Node*& head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* nextNode = curr->next;
        curr->next = prev;
    }
}
```

```

        prev = curr;
        curr = nextNode;
    }
    head = prev;
}

```

4.11 Reversal — Recursive O(n)

```

Node* reverseRecursive(Node* head) {
    if (head == nullptr || head->next == nullptr) return head;
    Node* newHead = reverseRecursive(head->next);
    head->next->next = head;
    head->next = nullptr;
    return newHead;
}

```

LeetCode equivalents: [206. Reverse Linked List](#) · [21. Merge Two Sorted Lists](#) · [141. Linked List Cycle](#) · [19. Remove Nth Node From End of List](#) · [876. Middle of the Linked List](#) · [83. Remove Duplicates from Sorted List](#)

5. Stack — Concept, Overflow/Underflow, Both Implementations

LIFO — Last In, First Out. Think of a stack of plates: you add/remove from the top only.

- **Overflow:** trying to push when the stack is full (array implementation only — a linked-list stack can't overflow unless memory runs out).
- **Underflow:** trying to pop/peek when the stack is empty.

Real-life examples: browser back button, undo (Ctrl+Z), function call stack during recursion, bracket matching in compilers.

5.1 Stack using Array

```

class ArrayStack {
    int* arr;
    int capacity;
    int top; // index of top element, -1 if empty

public:
    ArrayStack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }

    bool isEmpty() { return top == -1; }
    bool isFull() { return top == capacity - 1; }

    void push(int val) {
        if (isFull()) { cout << "Stack Overflow\n"; return; }
        arr[++top] = val;
    }

    int pop() {
        if (isEmpty()) { cout << "Stack Underflow\n"; return -1; }
        return arr[top--];
    }

    int peek() {
        if (isEmpty()) { cout << "Stack is empty\n"; return -1; }
        return arr[top];
    }
}

```

```

    }

    ~ArrayStack() { delete[] arr; }
};

```

5.2 Stack using Linked List

```

class LinkedStack {
    Node* top; // reuse Node struct from Section 4

public:
    LinkedStack() : top(nullptr) {}

    bool isEmpty() { return top == nullptr; }

    void push(int val) {
        Node* newNode = new Node(val);
        newNode->next = top;
        top = newNode; // O(1) - insertion always at head
    }

    int pop() {
        if (isEmpty()) { cout << "Stack Underflow\n"; return -1; }
        Node* temp = top;
        int val = temp->data;
        top = top->next;
        delete temp;
        return val; // O(1)
    }

    int peek() {
        if (isEmpty()) { cout << "Stack is empty\n"; return -1; }
        return top->data;
    }
};

```

Note why the linked-list stack always operates at the **head**, never the tail: insert/delete at head is $O(1)$, while at tail it's $O(n)$ — exactly matching the $O(1)$ requirement of push/pop.

5.3 Can a Stack Implement a Linked List? (relationship, not code)

The relationship only goes **one direction**:

- **Linked List → Stack**: yes, trivially — a stack is just a linked list with insert/delete restricted to the head (Section 5.2).
- **Stack → Linked List**: **no**. A linked list needs to insert/delete/search at *any* position. To do that with only a stack's top-access rule, you'd have to pop() every element above the target position, do the operation, then push() them all back — turning every operation into $O(n)$ with extra overhead and defeating the entire purpose of a linked list (efficient arbitrary-position access).

Exam framing: “A stack is a restricted-access linked list, not the other way around.”

LeetCode equivalents: [20. Valid Parentheses](#) · [155. Min Stack](#)

5.4 Bracket / Parenthesis Validation with Stack

```

bool isValidParentheses(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {

```

```

        if (st.empty()) return false;
        char top = st.top();
        if ((c == '(' && top != '(') ||
            (c == '[' && top != '[') ||
            (c == '{' && top != '{'))
            return false;
        st.pop();
    }
}
return st.empty();
}

```

6. String Reversal & Expression Conversion (Infix $\leftrightarrow$ Postfix $\leftrightarrow$ Prefix)

6.1 Reverse a String using a Stack

```

string reverseString(string s) {
    stack<char> st;
    for (char c : s) st.push(c);
    string result = "";
    while (!st.empty()) { result += st.top(); st.pop(); }
    return result;
}

```

6.2 Infix $\rightarrow$ Postfix (Shunting-Yard Algorithm)

Precedence: $\wedge > *, / > +, -$. $\wedge$ is right-associative; others left-associative.

```

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

string infixToPostfix(string infix) {
    stack<char> st;
    string postfix = "";
    for (char c : infix) {
        if (isdigit(c)) {
            postfix += c;
        } else if (c == '(') {
            st.push(c);
        } else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                postfix += st.top();
                st.pop();
            }
            st.pop(); // remove '('
        } else { // operator
            while (!st.empty() && precedence(st.top()) >=
precedence(c)
&& !(c == '^')) { // ^ is right-assoc, don't pop
equal-precedence ^
                postfix += st.top();
                st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { postfix += st.top(); st.pop(); }
    return postfix;
}

```

6.3 Infix → Prefix

Trick: reverse the infix string, swap (with), run infix-to-postfix, then reverse the result.

```
string infixToPrefix(string infix) {
    reverse(infix.begin(), infix.end());
    for (char &c : infix) {
        if (c == '(') c = ')';
        else if (c == ')') c = '(';
    }
    string postfix = infixToPostfix(infix);
    reverse(postfix.begin(), postfix.end());
    return postfix; // this is now prefix
}
```

LeetCode equivalents: [224. Basic Calculator](#) · [227. Basic Calculator II](#) · [1190. Reverse Substrings Between Each Pair of Parentheses](#) (stack + reversal)

7. Evaluation of Postfix, Prefix, Infix

7.1 Postfix Evaluation

Scan left to right: push numbers; on an operator, pop two, apply, push result.

```
int evaluatePostfix(string exp) {
    stack<int> st;
    for (char c : exp) {
        if (isdigit(c)) {
            st.push(c - '0');
        } else {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            switch (c) {
                case '+': st.push(a + b); break;
                case '-': st.push(a - b); break;
                case '*': st.push(a * b); break;
                case '/': st.push(a / b); break;
            }
        }
    }
    return st.top();
}
```

7.2 Prefix Evaluation

Scan **right to left**: push numbers; on an operator, pop two (first popped = a, second = b), compute a op b, push result.

```
int evaluatePrefix(string exp) {
    stack<int> st;
    for (int i = exp.size() - 1; i >= 0; i--) {
        char c = exp[i];
        if (isdigit(c)) {
            st.push(c - '0');
        } else {
            int a = st.top(); st.pop();
            int b = st.top(); st.pop();
            switch (c) {
                case '+': st.push(a + b); break;
                case '-': st.push(a - b); break;
                case '*': st.push(a * b); break;
                case '/': st.push(a / b); break;
            }
        }
    }
}
```

```

    }
    }
    return st.top();
}

```

7.3 Infix Evaluation

Simplest approach for exams: convert infix $\rightarrow$ postfix (Section 6.2), then evaluate postfix (Section 7.1). Complexity: $O(n)$ conversion + $O(n)$ evaluation = $O(n)$ overall.

LeetCode equivalent: [150. Evaluate Reverse Polish Notation](#) — exactly postfix evaluation.

8. Recursion

A function that calls itself needs:

1. **Base case** — stops the recursion.
2. **Recursive case** — reduces the problem toward the base case.

Every recursive call is pushed onto the **call stack**; when it hits the base case, calls resolve back down (unwind) — this is literally why Sections 5 and 8 are related (recursion *is* an implicit stack).

8.1 Factorial

```

int factorial(int n) {
    if (n <= 1) return 1;          // base case
    return n * factorial(n - 1);  // recursive case
}

```

8.2 Fibonacci (naive recursive)

```

int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

```

Complexity: **$O(2^n)$** — each call branches into two more calls (exponential, because of massive re-computation of the same subproblems).

8.3 Binary Search (recursive)

```

int binarySearch(int arr[], int low, int high, int key) {
    if (low > high) return -1; // base case: not found
    int mid = low + (high - low) / 2;
    if (arr[mid] == key) return mid;
    else if (arr[mid] < key) return binarySearch(arr, mid + 1, high,
key);
    else return binarySearch(arr, low, mid - 1, key);
}

```

Complexity: **$O(\log n)$** — search space halves each call.

8.4 GCD (Euclidean Algorithm)

```

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

```

8.5 Tower of Hanoi

```
void hanoi(int n, char from, char to, char aux) {
    if (n == 0) return;
    hanoi(n - 1, from, aux, to);
    cout << "Move disk " << n << " from " << from << " to " << to <<
endl;
    hanoi(n - 1, aux, to, from);
}
```

Complexity: $O(2^n)$.

8.6 Recursion + Linked List

Recursive traversal:

```
void printRecursive(Node* head) {
    if (head == nullptr) return;
    cout << head->data << " ";
    printRecursive(head->next);
}
```

Recursive length:

```
int lengthRecursive(Node* head) {
    if (head == nullptr) return 0;
    return 1 + lengthRecursive(head->next);
}
```

Recursive reversal was already shown in Section 4.11 — that’s the classic “recursion + linked list” exam question.

8.7 Recursion + Stack

Reverse a stack using recursion only (no extra stack/array — a very common viva question):

```
// Step 1: insert an element at the BOTTOM of the stack recursively
void insertAtBottom(stack<int>& st, int val) {
    if (st.empty()) { st.push(val); return; }
    int top = st.top();
    st.pop();
    insertAtBottom(st, val);
    st.push(top);
}

// Step 2: reverse the whole stack using insertAtBottom
void reverseStack(stack<int>& st) {
    if (st.empty()) return;
    int top = st.top();
    st.pop();
    reverseStack(st);
    insertAtBottom(st, top);
}
```

This works because the **call stack itself acts as temporary storage** — a nice concrete example of “recursion is a hidden stack.”

8.8 More Famous Recursive Problems (bonus, likely to appear)

```
// Power function: x^n
double power(double x, int n) {
    if (n == 0) return 1;
    if (n < 0) return 1 / power(x, -n);
    double half = power(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
```

```

    } // O(log n)

    // Sum of digits
    int sumOfDigits(int n) {
        if (n == 0) return 0;
        return (n % 10) + sumOfDigits(n / 10);
    }

    // Check if array is sorted (recursive)
    bool isSorted(int arr[], int n) {
        if (n <= 1) return true;
        return (arr[n-1] >= arr[n-2]) && isSorted(arr, n-1);
    }

```

LeetCode equivalents: [509. Fibonacci Number](#) · [704. Binary Search](#) · [70. Climbing Stairs](#) (Fibonacci pattern) · [50. Pow\(x, n\)](#) · [206. Reverse Linked List](#) (recursive version) · [1290. Convert Binary Number in a Linked List to Integer](#) (recursion + linked list) · [1614. Maximum Nesting Depth of the Parentheses](#) (stack-depth reasoning, ties to recursion depth)

9. LeetCode Solutions — Every Problem Referenced Above

Each entry: **problem + link**, **topic it falls under**, a one-line description, the C++ solution, and complexity.

9.1 Arrays / 2D Arrays / Binary Search

1. Two Sum — *Topic: Arrays* Given an array and a target, return the indices of the two numbers that add up to it.

```

vector<int> twoSum(vector<int>& nums, int target)
{
    unordered_map<int, int> seen;
    for (int i = 0; i < nums.size(); i++)
    {
        int need = target - nums[i];
        if (seen.count(need))
            return {seen[need], i};
        seen[nums[i]] = i;
    }
    return {};
}

```

Complexity: O(n) time, O(n) space.

54. Spiral Matrix — *Topic: 2D Arrays* Return all elements of an m×n matrix in spiral order.

```

vector<int> spiralOrder(vector<vector<int>>& matrix)
{
    vector<int> result;
    if (matrix.empty()) return result;

    int top = 0, bottom = matrix.size() - 1;
    int left = 0, right = matrix[0].size() - 1;

    while (top <= bottom && left <= right)
    {
        for (int j = left; j <= right; j++)
            result.push_back(matrix[top][j]);
        top++;

        for (int i = top; i <= bottom; i++)

```

```

result.push_back(matrix[i][right]);
    right--;

    if (top <= bottom)
    {
        for (int j = right; j >= left; j--)
            result.push_back(matrix[bottom][j]);
        bottom--;
    }

    if (left <= right)
    {
        for (int i = bottom; i >= top; i--)
            result.push_back(matrix[i][left]);
        left++;
    }
}
return result;
}

```

Complexity: $O(m \times n)$ time, $O(1)$ extra space.

74. Search a 2D Matrix — *Topic: 2D Arrays + Binary Search* Rows are sorted and each row's first element exceeds the previous row's last — treat the whole grid as one flattened sorted array.

```

bool searchMatrix(vector<vector<int>>& matrix, int target)
{
    int rows = matrix.size(), cols = matrix[0].size();
    int low = 0, high = rows * cols - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        int val = matrix[mid / cols][mid % cols];

        if (val == target) return true;
        else if (val < target) low = mid + 1;
        else high = mid - 1;
    }
    return false;
}

```

Complexity: $O(\log(m \times n))$.

35. Search Insert Position — *Topic: Binary Search* Return the index of the target, or where it should be inserted to keep the array sorted.

```

int searchInsert(vector<int>& nums, int target)
{
    int low = 0, high = nums.size() - 1;
    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return low;
}

```

Complexity: $O(\log n)$.

704. Binary Search — *Topic: Binary Search / Recursion* Find target in a sorted array (iterative version; recursive version already in Section 8.3).

```
int search(vector<int>& nums, int target)
{
    int low = 0, high = nums.size() - 1;
    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Complexity: $O(\log n)$.

9.2 Linked List

206. Reverse Linked List — *Topic: Linked List* Reverse a singly linked list in place.

```
ListNode* reverseList(ListNode* head)
{
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr != nullptr)
    {
        ListNode* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

21. Merge Two Sorted Lists — *Topic: Linked List* Merge two already-sorted linked lists into one sorted list.

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2)
{
    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (l1 != nullptr && l2 != nullptr)
    {
        if (l1->val <= l2->val)
        {
            tail->next = l1;
            l1 = l1->next;
        }
        else
        {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = (l1 != nullptr) ? l1 : l2;
    return dummy.next;
}
```

Complexity: $O(n + m)$.

141. Linked List Cycle — *Topic: Linked List* Detect whether a linked list has a cycle, using Floyd's slow/fast pointer technique.

```
bool hasCycle(ListNode* head)
{
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr)
    {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Complexity: O(n) time, O(1) space.

19. Remove Nth Node From End of List — *Topic: Linked List*

Remove the n-th node from the end in a single pass using a two-pointer gap.

```
ListNode* removeNthFromEnd(ListNode* head, int n)
{
    ListNode dummy(0);
    dummy.next = head;
    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int i = 0; i < n; i++) fast = fast->next;

    while (fast->next != nullptr)
    {
        fast = fast->next;
        slow = slow->next;
    }

    ListNode* toDelete = slow->next;
    slow->next = slow->next->next;
    delete toDelete;
    return dummy.next;
}
```

Complexity: O(n).

876. Middle of the Linked List — *Topic: Linked List*

Return the middle node (slow/fast pointer — fast moves twice as far as slow).

```
ListNode* middleNode(ListNode* head)
{
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Complexity: O(n).

83. Remove Duplicates from Sorted List — *Topic: Linked List*

Given a sorted linked list, delete duplicate values so each element appears once.

```
ListNode* deleteDuplicates(ListNode* head)
{
    ...
}
```

```

ListNode* curr = head;
while (curr != nullptr && curr->next != nullptr)
{
    if (curr->val == curr->next->val)
    {
        ListNode* toDelete = curr->next;
        curr->next = curr->next->next;
        delete toDelete;
    }
    else
    {
        curr = curr->next;
    }
}
return head;
}

```

Complexity: $O(n)$.

9.3 Stack

20. Valid Parentheses — *Topic: Stack* Check whether every opening bracket has a matching, correctly-ordered closing bracket. (Full solution already in Section 5.4 — same code applies here.) Complexity: $O(n)$.

155. Min Stack — *Topic: Stack* Design a stack where push, pop, top, and getMin all run in $O(1)$. Track the running minimum in a parallel stack.

```

class MinStack
{
    stack<int> st;
    stack<int> minSt;

public:
    void push(int val)
    {
        st.push(val);
        if (minSt.empty() || val <= minSt.top())
            minSt.push(val);
        else
            minSt.push(minSt.top());
    }

    void pop()
    {
        st.pop();
        minSt.pop();
    }

    int top()
    {
        return st.top();
    }

    int getMin()
    {
        return minSt.top();
    }
};

```

Complexity: $O(1)$ per operation, $O(n)$ space.

224. Basic Calculator — *Topic: Stack / Expression Evaluation* Evaluate a string expression containing +, -, parentheses, and non-negative integers.

```

int calculate(string s)
{
    stack<int> st;
    int result = 0;
    int number = 0;
    int sign = 1;

    for (char c : s)
    {
        if (isdigit(c))
        {
            number = number * 10 + (c - '0');
        }
        else if (c == '+')
        {
            result += sign * number;
            number = 0;
            sign = 1;
        }
        else if (c == '-')
        {
            result += sign * number;
            number = 0;
            sign = -1;
        }
        else if (c == '(')
        {
            st.push(result);
            st.push(sign);
            result = 0;
            sign = 1;
        }
        else if (c == ')')
        {
            result += sign * number;
            number = 0;
            result *= st.top(); st.pop();
            result += st.top(); st.pop();
        }
    }
    return result + sign * number;
}

```

Complexity: $O(n)$.

227. Basic Calculator II — Topic: Stack / Expression Evaluation

Evaluate a string with +, -, *, / (no parentheses), respecting operator precedence.

```

int calculateII(string s)
{
    stack<int> st;
    int number = 0;
    char op = '+';

    for (int i = 0; i < s.size(); i++)
    {
        char c = s[i];
        if (isdigit(c)) number = number * 10 + (c - '0');

        if ((!isdigit(c) && c != ' ') || i == s.size() - 1)
        {
            if (op == '+') st.push(number);
            else if (op == '-') st.push(-number);
            else if (op == '*')
            {
                int top = st.top(); st.pop();
                st.push(top * number);
            }
        }
    }
}

```

```

    }
    else if (op == '/')
    {
        int top = st.top(); st.pop();
        st.push(top / number);
    }
    op = c;
    number = 0;
}
}

int result = 0;
while (!st.empty())
{
    result += st.top();
    st.pop();
}
return result;
}

```

Complexity: $O(n)$.

1190. Reverse Substrings Between Each Pair of Parentheses —

Topic: Stack Reverse the string inside every pair of parentheses, innermost first, then remove the brackets.

```

string reverseParentheses(string s)
{
    stack<string> st;
    string current = "";

    for (char c : s)
    {
        if (c == '(')
        {
            st.push(current);
            current = "";
        }
        else if (c == ')')
        {
            reverse(current.begin(), current.end());
            current = st.top() + current;
            st.pop();
        }
        else
        {
            current += c;
        }
    }
    return current;
}

```

Complexity: $O(n^2)$ worst case (string rebuilding).

150. Evaluate Reverse Polish Notation — *Topic: Stack / Postfix*

Evaluation Evaluate an expression already given in postfix (RPN) form — exactly Section 7.1's algorithm.

```

int evalRPN(vector<string>& tokens)
{
    stack<int> st;
    for (string& token : tokens)
    {
        if (token == "+" || token == "-" || token == "*" || token ==
"/")
        {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();

```

```

        if (token == "+") st.push(a + b);
        else if (token == "-") st.push(a - b);
        else if (token == "*") st.push(a * b);
        else st.push(a / b);
    }
    else
    {
        st.push(stoi(token));
    }
}
return st.top();
}

```

Complexity: $O(n)$.

9.4 Recursion

509. Fibonacci Number — *Topic: Recursion* Return the n -th Fibonacci number.

```

int fib(int n)
{
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

```

Complexity: $O(2^n)$ naive. A memoized version fixes this:

```

int fibMemo(int n, vector<int>& memo)
{
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

```

Complexity (memoized): $O(n)$.

70. Climbing Stairs — *Topic: Recursion / DP* Count the ways to climb n stairs taking 1 or 2 steps at a time — same recurrence as Fibonacci.

```

int climbStairs(int n)
{
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++)
    {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

```

Complexity: $O(n)$ time, $O(1)$ space (the naive fully-recursive version is $O(2^n)$, same problem as raw Fibonacci).

50. Pow(x, n) — *Topic: Recursion* Compute x^n using fast exponentiation (halving n each step, as in Section 8.8).

```

double myPow(double x, long long n)
{
    if (n < 0)
    {
        x = 1 / x;
        n = -n;
    }

    double result = 1;

```

```

while (n > 0)
{
    if (n % 2 == 1) result *= x;
    x *= x;
    n /= 2;
}
return result;
}

```

Complexity: $O(\log n)$.

206. Reverse Linked List — recursive version — *Topic: Recursion + Linked List* Already given in Section 4.11 — the same code solves this LeetCode problem directly. Complexity: $O(n)$ time, $O(n)$ call-stack space.

1290. Convert Binary Number in a Linked List to Integer — *Topic: Recursion + Linked List* Each node holds a 0 or 1 (most significant bit first); return the decimal value.

```

int traverse(ListNode* node, int result)
{
    if (node == nullptr) return result;
    return traverse(node->next, result * 2 + node->val);
}

int getDecimalValue(ListNode* head)
{
    return traverse(head, 0);
}

```

Complexity: $O(n)$.

1614. Maximum Nesting Depth of the Parentheses — *Topic: Stack / Recursion Depth* Find the maximum depth of nested parentheses — conceptually the maximum size a validation stack (Section 5.4) would ever reach, or equivalently the deepest a recursive descent parser would go.

```

int maxDepth(string s)
{
    int depth = 0, maxD = 0;
    for (char c : s)
    {
        if (c == '(')
        {
            depth++;
            maxD = max(maxD, depth);
        }
        else if (c == ')')
        {
            depth--;
        }
    }
    return maxD;
}

```

Complexity: $O(n)$.

Quick Revision Table — Everything in One Glance

Topic	Key Idea	Typical Complexity
	Behavior contract,	

ADT	implementation-independent	—
Row/Col major	$B + (i * COLS + j) * w$ vs $B + (j * ROWS + i) * w$	$O(1)$ address calc
Big $O/\Omega/\Theta$	Worst/Best/Tight bound	—
Array insert/delete at start	Shifting required	$O(n)$
Array insert/delete at end	No shifting (if space)	$O(1)$
LL insert/delete at head	Just relink	$O(1)$
LL insert/delete at tail	Traverse to find it	$O(n)$
Stack push/pop (array or LL)	Always operate at one end	$O(1)$
Infix→Postfix/Prefix	Stack-based, precedence-driven	$O(n)$
Postfix/Prefix evaluation	Single stack pass	$O(n)$
Recursion base case	Must shrink toward it every call	Depends ($O(n)$, $O(\log n)$, $O(2^n)$...)
Fibonacci (naive recursive)	Recomputes subproblems	$O(2^n)$
Binary search (recursive)	Halves search space	$O(\log n)$
